

1 Les bases de python3

Les types

Les différents types entiers, flottants, complexes, chaînes de caractères, booléens, listes, tuples et dictionnaires.

- *entier* : 3
- *flottant* : 2.3
- *complexe* : $1+2j$
- *chaînes de caractères (string)* : 'ISN'
- *Boléen* : True
- *Listes* : [0,1,2,3]
- *Byte* : b'toto'
- On aura toujours à l'esprit que $0.2 + 0.7 = 0.8999999999999999 \neq 0.9$
- *int()* permet de convertir, un nombre ou une chaîne de caractère en un entier.
- *float()* permet de convertir, un nombre ou une chaîne de caractère en un flottant.
- On utilise le caractère # pour écrire un commentaire

Quelques opérateur déjà vu et très utiles

Les opérateurs

Opération	algorithmique	python
Addition	$2+3$	$2+3$
Soustraction	$12-5$	$12-5$
Multiplication	$3*6$	$3*6$
Division	$7/2$	$7/2$
Quotient de la division euclidienne	$7 \text{ div } 2$ ou $\text{div}(7,2)$	$7//2$
Reste de la division euclidienne	$7 \bmod 2$ ou $\text{mod}(7,2)$	$7\%2$
puissance	7^2	$7**2$
racine carrée	$\sqrt{2}$ ou $\text{sqrt}(2)$ du module <i>maths</i>	$\text{sqrt}(2)$

Variables

- Python utilise le symbole = pour affecter une valeur à une variable.
- Attention à ne pas confondre $a = 12$ et $a == 12$.

```
A=3; B=5
C='toto'
```

Entrées

- *input("texte")* permet de saisir du texte pour un programme.
- Il faudra éventuellement convertir ce texte dans le type voulu avec *int()* ou *float()*

```
a=input("Entrer un texte a: ")
b=int(input("Entrer un entier b: "))
c=float(input("Entrer un réel c: "))
```

Affichage

- On utilise de préférence les *f-string* pour afficher du texte et des variables.

```
print("La valeur de la variable a est",a, ".") # si a est un nombre
print("La valeur de la variable a est " + str(a) + ".") # autre forme
print(f"La valeur de la variable a est {a}.") # je préfère
print(f"Le produit de {a} par {b} est {a*b}")
```

Fonction

Syntaxe :

Fonction affine $f : x \mapsto 3x + 1$ et afficher les valeurs.

```
def nomfonction(parametres):
    instructions
    instructions
    return valeur
```

```
def f(x):
    return 3*x+1
for i in range(-5,6):
    print(f(i))
```

Connecteurs logiques

algorithmique	python
$a = b$	$a==b$
$a \neq b$	$a!=b$
A et B	A and B A & B

algorithmique	python
A ou B	A or B $A \mid B$
A xor B	$A \wedge B$
non A	not(A)

Condition SI

```
if x<=0:
    print('Pas de racines')
```

```
if x<=0:
    print('Pas de racines')
else:
    print('1 ou 2 racines')
```

```
if x<=0:
    print('Pas de racines')
elif x==0:
    print('1 racines double')
else:
    print('2 racines distinctes')
```

```
print("Saisissez deux valeurs numériques")
a=float(input("Saisir a: "))
b=float(input("Saisir b: "))
if a==b :
    print("Vous avez saisi deux fois la même valeur, à savoir {}".format(a))
else :
    print("Vous avez saisi deux valeurs différentes {} et {}".format(a,b))
```

Boucle pour

```
for i in range(4,21,2): # 4 6 ... 20
    print(i)
for i in range(6):      # de 0 à 5 sinon utiliser range(1,7) pour 1 2 ... 6
    print(i)
```

La syntaxe générale est **for i in range(m,n,p)** :

i prend alors toutes les valeurs de **m** à **n-1** par pas de **p**

Tant que

```
i=1
while i<=5:
    print(i)
    i=i+1 #où en plus concis i+=1
#À la sortie de la boucle i=6
```

2 Les Tableaux

- `tab[10]` permet d'accéder à l'élément d'indice 10 du tableau `tab`.
- `tab[-1]` est le dernier élément d'un tableau.
- Le premier indice d'un tableau est 0.
- `[0]*1000` permet de construire un tableau de 1000 zéro (avec des indices de 0 à 999).
- `len(tab)` renvoie la taille du tableau `tab`.
- `tab=[1,2,3]` la valeur affectée à la variable `tab` n'est pas un tableau mais l'adresse mémoire de l'espace alloué au tableau.
- Il y a deux méthodes pour parcourir un tableau
 - Méthode 1 : Cette méthode nous permet de parcourir tous les éléments d'un tableau

```
tab=[3,1,8,4,6]
for valeur in tab:
    print(valeur)
```

- Méthode 2 : Cette méthode nous permet de parcourir tous les éléments du tableau et d'avoir accès aux indices.

```
tab=[3,1,8,4,6]
for i in range(len(tab)):
    print(f"L'élément d'indice {i} est {tab[i]}")
```

3 html

Le **World Wide Web**, que l'on appelle le plus souvent **Web**, est un des services disponibles sur Internet.

D'autres services disponibles sur Internet sont par exemple la messagerie électronique (**email**) ou le transfert de fichier (**FTP**)

```
<!DOCTYPE html>
<html>
<head>
  <!-- En-tête de la page -->
  <meta charset="utf-8" />
  <title>Titre</title>
</head>
<body>
  <!-- Corps de la page -->
</body>
</html>
```

Voici quelques balises à connaître

- `<p> ... </p>` pour encadrer un paragraphe
- `<br />` pour sauter une ligne
- `<h1> ... </h1>` : signifie « titre très important ».
- `<h2> ... </h2>` : signifie « titre important ».
- `<h3> ... </h3>` : pareil, c'est un titre un peu moins important. etc...
- Listes à puces (*unordonnées*)

```
<ul>
  <li> Fraises </li>
  <li> Framboises </li>
  <li> Cerises </li>
</ul>
```

- Listes numérotées (*ordonnées*)

```
<ol>
  <li> Fraises </li>
  <li> Framboises </li>
  <li> Cerises </li>
</ol>
```

- Un tableau

```
<table>
  <tr><th>Nom</th><th>prénom</th><th>âge</th><
  /tr>
  <tr><td>Onette</td><td>Camille</td><td>15</
  td></tr>
  <tr><td>Vanille</td><td>Douglas</td><td>16</
  td></tr>
  <tr><td>Bijoba</td><td>Joe</td><td>16</td></
  tr>
</table>
```

- Une image

```
<img alt='Pablo Neruda' src='https://upload.
wikimedia.org/wikipedia/commons/8/86/
Pablo_Neruda_1963.jpg' width=200px />
<img alt='Texte alternatif' src='Monimage.jpg'
width=200px />
```

- Des liens

```
<h1>Les liens</h1>
<p>Cliquez sur <a href="http://wikipedia.org">
ce lien externe</a>pour aller sur Wikipé
dia.</p>
<p>Cliquez sur <a href="page2.html">ce lien
interne</a>pour aller sur une deuxième
page html.</p>
```

4 Javascript

- `//... Pour un commentaire sur une ligne`
- `/* ... */ pour un commentaire sur plusieurs lignes.`

- Pour saisir du texte `var prenom = prompt('Quel est votre prénom ?');`

- Pour afficher contenu d'une variable :

```
alert('Bonjour ' + prenom) par console.log('Bonjour ' + prenom);
console: Affiche le résultat dans la console log.
console.log('Bonjour ' + prenom);
```

Seulement dans les notebook

```
element.append('Bonjour ' + prenom + "<br>");
element.text('Bonjour ' + prenom + "<br>");
```

- Les variables : `var nombre = 3;`

- Les tableaux `var tab = ['Marius', 'Elric', 'El-Tahar', 'Chahine', 'Chloé'];`

- Les principales méthodes sont :

- `.find(element)` renvoie la valeur du premier élément trouvé dans le tableau.
- `.pop()` supprime le dernier élément d'un tableau et retourne cet élément
- `.push()` ajoute un ou plusieurs éléments à la fin d'un tableau et retourne la nouvelle taille du tableau.
- `.shift()` permet de retirer le premier élément d'un tableau et de renvoyer cet élément.
- `.slice()` permet de faire un slicing.

- Condition si

```
if (condition) {
    # Traitement bloc 1
}
else {
    # Traitement bloc 2
}
```

- Boucle while

```
var u = 0.1;
var i = 0
while (u <= 10) {
    u=u*1.02;
    i++
}
alert("u("+i+")="+u)
```

- Boucle for

```
var i = 1;
var u = 1;
var text=""
for (i = 0; i <= 10; i++) {
    u=u+3
    text.append( "u("+i+")=" + u + "<br>");
}
alert(text)
```

- Sélectionner par son attribut id : `var element = document.getElementById("idun");`

- par son attribut class `var classList = document.getElementsByClassName("uneclass");`

- par sa balise `var divList = document.getElementsByTagName("div");`

- par un sélecteur CSS

```
//Tous les éléments
var listElement = document.querySelectorAll("div.exercice img");
// le premier de ces éléments
var premier=document.querySelector("div.exercice img");
```

5 Les principales méthodes

5.1 Méthodes des listes

- ***l.append(x)*** : Ajoute x à la fin de la liste.

```
>>> l=[1,2,3,4,5]
>>> l.append(6)
>>> l
[1, 2, 3, 4, 5, 6]
```

- ***l.clear()*** Efface la liste l (*depuis python3.3*).

```
>>> l.clear()
>>> l
[]
```

- ***l.extend (t)*** : concatène la liste l avec le contenu de la liste t sans réaffectation.

```
>>> t=(2,3,4,2)
>>> l1=[1,2,3]
>>> l2=[4,5,6]
>>> l1.extend(l2)
>>> l1
[1, 2, 3, 4, 5, 6]
```

- ***l.insert(n,x)*** : insert x à la liste l à l'indice n.

```
>>> l=[1,2,3,4]
>>> l.insert(1,8)
>>> l
[1, 8, 2, 3, 4]
```

- ***l.pop ([i])*** : affiche l'élément d'indice i et aussi supprime cet élément de l.

```
>>> l=[1,2,3,1]
>>> a=l.pop(2)
>>> a
3
>>> l
[1, 2, 1]
```

- ***l.remove (x)*** : supprime le premier élément x de s et affiche une *ValueError* s'il ne le trouve pas.

```
>>> l=[1,2,3,1]
>>> l.remove(1)
>>> l
[2, 3, 1]
>>> l.remove(1)
>>> l
[2, 3]
>>> l.remove(1)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: list.remove(x): x not in list
```

- ***l.reverse()*** inverse les éléments de l.

```
>>> l=[1,2,3]
>>> l.reverse()
>>> l
[3, 2, 1]
```

- ***l.sort()*** Trie les éléments de l.

```
>>> l=[4,6,2,1,0,5,3]
>>> l.sort()
>>> l
[0, 1, 2, 3, 4, 5, 6]
```

5.2 Méthodes des chaîne de caractères

- ***str.split()*** : Retourne une liste des mots de la chaîne, en utilisant *sep* pour séparateur.

```
>>> 'Pablo Neruda'.split()
['Pablo', 'Neruda']
>>> 'Pablo Neruda'.split(' ')
['Pablo', 'Neruda']
>>> 'Ligne 1\nligne2'.split('\n')
['Ligne 1', 'ligne2']
```

Il existe aussi une variante avec *rsplit()*, (r pour right) la différence se produit lorsque le deuxième argument est spécifié.

```
>>> 'Pablo Neruda Meilland jean claudé'.rsplit(' ')
['Pablo', 'Neruda', 'Meilland', 'jean', 'claudé']
>>> 'Pablo Neruda Meilland jean claudé'.split(' ')
['Pablo', 'Neruda', 'Meilland', 'jean', 'claudé']
```

- ***str.join(iterable)*** : Retourne une chaîne qui est la concaténation de str dans iterable

```
>>> '-'.join('abcd')
'a-b-c-d'
>>> ''.join(['a','e','i','o','u','y'])
'aeiouy'
```

- ***len()*** compte le nombre de caractère de la chaîne.

```
>>> len('Pablo Neruda')
12
```

- ***str.count(sub[, start[, end]])*** : Retourne le nombre d'occurrences du caractère *sub* dans l'intervalle *[start;end]*.

```
>>> 'Pablo Neruda'.count('a')
2
```

- ***str.find(sub[, start[, end]])*** : Retourne l'indice le plus faible dans la chaîne où *sub* se trouve. Retourne -1 si *sub* n'est pas trouvé.

```
>>> 'Pablo Neruda'.find('a')
1
>>> 'Pablo Neruda'.find('ll')
-1
```

- ***str.index(sub[, start[, end]])*** : Comme *find()* , mais retourne une *ValueError* quand la sous-chaîne n'est pas trouvée.
- ***str.rfind(sub[, start[, end]])*** : Retourne l'indice le plus élevé dans la chaîne où *sub* se trouve. Retourne -1 en cas d'échec.

```
>>> 'Pablo Neruda'.rfind('a')
11
```

- ***str.replace(old, new[, count])*** : Retourne une copie de la chaîne de toutes les occurrences de la chaîne *old* remplacées par de *new*

```
>>> 'Pablo Neruda'.replace('a','*')
'P*blo Nerud*'
>>> 'Pablo Neruda'.replace('a','*',1)
'P*blo Neruda'
```

- **sub in str** : Retourne *True* si *sub* est une sous-chaîne de *str* *False* sinon.

```
>>> 'a' in 'Pablo Neruda'
True
```

- **str.strip([chars])** : Retourne une copie de la chaîne avec les avant et arrière caractères supprimés. L'argument de *chars* est une chaîne spécifiant le jeu de caractères à supprimer (espace par défaut).

```
>>> '  Pablo Neruda  '.strip()
'Pablo Neruda'
```

5.3 Méthodes des tuples

- **t.count(x)** : Retourne le nombre d'occurrences de *x* dans le tuple *t*.

```
>>> t=(2,3,2,4,2)
>>> t.count(2)
3
```

- **t.index(x)** : Retourne l'indice le plus faible dans le tuple *t* où *x* se trouve. retourne une *ValueError* quand *x* n'est pas trouvée.

```
>>> t=(2,3,4,2)
>>> t.index(2)
0
```

6 P-uplets nommés (dictionnaires)

6.1 Créer un dictionnaire

```
>>> dico1 = {"nom": "simpson", "prénom": "Homer", "année": 1989}
>>> dico1
{'nom': 'simpson', 'prénom': 'Homer', 'année': 1989}
```

```
>>> dico2 = {} # Un dictionnaire vide
>>> dico2["nom"] = "simpson"
>>> dico2["prénom"] = "Homer"
>>> dico2["année"] = 1989
>>> dico2
{'nom': 'simpson', 'prénom': 'Homer', 'année': 1989}
```

```
>>> dico3 = {}
>>> dico3.update({"nom": "simpson", "prénom": "Homer", "année": 1989})
>>> dico3
{'nom': 'simpson', 'prénom': 'Homer', 'année': 1989}
```

```
>>> liste = [{"nom", "simpson"}, {"prénom", "Homer"}, {"année", 1989}]
>>> dico4 = dict(liste)
>>> dico4
{'nom': 'simpson', 'prénom': 'Homer', 'année': 1989}
```

```
>>> liste_keys = ["nom", "prénom", "année"]
>>> liste_values = ["simpson", "Homer", 1989]
>>> dico5 = {key: value for key, value in zip(liste_keys, liste_values)}
>>> dico5
{'nom': 'simpson', 'prénom': 'Homer', 'année': 1989}
```

6.2 Lire et modifier des valeurs des valeurs

```
>>> print(dico1["nom"])
simpson
>>> "surnom" in dico1 # teste si "surnom" est une clé
False
>>> "Homer" in dico1.values() # teste si Homer est une valeur
True
>>> dico1["année"]=1990 # On change l'année de naissance
>>> dico1
{'nom': 'simpson', 'prénom': 'Homer', 'année': 1990}
```

6.3 Parcourir un dictionnaire

```
>>> for cle in dico1.keys(): # ou simplement for key in dico1.keys():
...     print(f"la clé est {cle} et sa valeur est {dico1[cle]}")
...
la clé est nom et sa valeur est simpson.
la clé est prénom et sa valeur est Homer.
la clé est année et sa valeur est 1990.
```

```
>>> for valeur in dico1.values():
...     print(valeur)
...
simpson
Homer
1990
```

```
>>> for key,value in dico1.items():
...     print(f"La clé est {key} et sa valeur {value}")
...
La clé est nom et sa valeur simpson
La clé est prénom et sa valeur Homer
La clé est année et sa valeur 1990
```

6.4 Suppression d'un élément

```
>>> dico1 = {"nom": "simpson", "prénom": "Homer", "année": 1989}
>>> del dico1["prénom"]
>>> dico1
{'nom': 'simpson', 'année': 1989}
```

```
>>> dico1 = {"nom": "simpson", "prénom": "Homer", "année": 1989}
>>> dico1.pop('nom')
'simpson'
>>> dico1
{'prénom': 'Homer', 'année': 1989}
```

```
>>> dico1 = {"nom": "simpson", "prénom": "Homer", "année": 1989}
>>> dico1.popitem()
('année', 1989)
>>> dico1
{'nom': 'simpson', 'prénom': 'Homer'}
```

6.5 Les méthodes

- `.copy()` : Copie d'un dictionnaire car une variable contient l'adresse d'un dictionnaire et non le dictionnaire lui-même.
- `.fromkeys(liste_cles, valeur)` : Crée un nouveau dictionnaire avec les clefs de `liste_cles` et la `valeur`.

```
>>> dico1 = {"nom": "simpson", "prénom": "Homer", "année": 1989}
>>> liste_cles=['nom','prenom','années']
>>> dico1.fromkeys(liste_cles,0)
{'nom': 0, 'prenom': 0, 'années': 0}
```

- `.pop(key[, default])` : Supprime *key* si *key* est dans le dictionnaire sinon renvoie default.

```
>>> dico1 = {"nom": "simpson", "prénom": "Homer", "année": 1989}
>>> var=dico1.pop('nom')
>>> print(var)
simpson
>>> print(dico1)
{'prénom': 'Homer', 'année': 1989}
```

- `.popitem()` : Supprime et renvoie une paire (*key*, *value*) du dictionnaire.

```
>>> dico1 = {"nom": "simpson", "prénom": "Homer", "année": 1989}
>>> var=dico1.popitem()
>>> print(var)
('année', 1989)
>>> print(dico1)
{'nom': 'simpson', 'prénom': 'Homer'}
```

- `.keys()` Les clés.
- `.values()` Les valeurs.
- `.items()` (*clés*, *valeurs*).

7 Fichier texte

- Télécharger un fichier

```
import urllib.request
url = 'https://megamaths.hd.free.fr/static/texte2.txt'
urllib.request.urlretrieve(url, './texte2.txt')
```

- Ouverture en lecture(read)

```
monfichier = open('lenom.txt', 'r', encoding='latin1')
```

```
with open('lenom.txt', 'r') as f:
    ...
```

- Ouverture en écriture (write)

```
monfichier = open('lenom.txt', 'w', encoding='utf8')
```

```
with open('lenom.txt', 'w') as f:
    ...
```

- Ouverture et ajouter du texte (append).

```
monfichier = open('lenom.txt', 'a')
```

```
with open('lenom.txt', 'a') as f:
    ...
```

- Parcourir toutes les lignes

```
for ligne in monfichier.readlines():
    ...
```

- Lire la ligne suivante

```
ligne = monfichier.readline()
```

- Lire toutes les lignes

```
txt=monfichier.read()
```

- Fermeture

```
monfichier.close()
```

- Séparer une chaîne selon les symboles ';' ;

```
ligne.split(';')
```

- Supprimer les blancs (et saut de ligne) en début et en fin de ligne

```
ligne.strip()
```

8 csv

- Si on dispose du fichier *countries.csv* :

```
import csv
with open('countries.csv', newline='') as csvfile:
    csvreader = csv.DictReader(csvfile, delimiter=';')
    for row in csvreader:
        pays.append(row)
```

- Si on parse le fichier *countries.csv* :

```
import csv
import urllib.request
import codecs

url = "https://megamaths.hd.free.fr/static/countries.csv"
htpstream = urllib.request.urlopen(url)
csvfile = csv.DictReader(codecs.iterdecode(htpstream, 'utf-8'), delimiter=';')
pays = []
for row in csvfile:
    pays.append(dict(row))
```

- Un tri selon des critères

```
def cle_population(p):
    return int(p['population'])

def cle_continent(p):
    return p['continent']

pays.sort(key=cle_population, reverse=True)
pays.sort(key=cle_continent)
print(pays) # la liste est triée par continents du pays le plus peuplé au moins peuplé.
```